
CHAPITRE 4

Programmation et conception objet

Comprendre les objets Python, les utiliser et les faire évoluer

Comment travailler ce chapitre

Le chapitre suppose que vous savez écrire une fonction, parcourir une liste et utiliser `if`, `return` et `try/except`.

Nous commencerons par les classes et leur initialisation. Nous verrons ensuite comment contrôler les attributs, utiliser l'héritage et faire travailler plusieurs objets ensemble. La dernière partie reprend ces idées pour préparer des données et utiliser un modèle.

Pour les exercices : écrivez le pseudocode avant le programme. Quand une boucle intervient, tracez ses variables à la main, puis comparez avec Python. Les corrigés suivent les exercices.

Conservez les classes dans un même fichier ou notebook. Les ajouts de méthodes sont signalés. Après avoir modifié une classe, exécutez sa définition et recréez les objets pour utiliser la nouvelle version.

Les quatre notions de base

Un **objet** regroupe des données et des opérations. Une **classe** définit un type d'objet. Un objet créé à partir de cette classe est une **instance**.

Notion	Signification	Exemple
Classe	Définit les objets que l'on peut créer	<code>list</code> est la classe des listes
Instance	Un objet particulier de cette classe	La liste <code>[2, 4]</code>
Attribut	Une donnée accessible par un nom	Bientôt : <code>produit.prix</code>
Méthode	Une fonction appelée sur un objet	<code>valeurs.append(6)</code>

Les valeurs des attributs constituent l'**état** de l'objet. La notation avec un point permet d'accéder à un attribut ou à une méthode. `class Nom:` commence une définition de classe. Son contenu est indenté.

La création d'un objet et `__init__`

`__init__` est une **méthode spéciale d'initialisation** : Python l'appelle automatiquement lorsqu'on crée une instance. Son premier paramètre, nommé `self` par convention, désigne cette instance.

```
class Compteur:
    def __init__(self, depart):
        self.valeur = depart

c = Compteur(3)
print(c.valeur)
```

`Compteur(3)` crée un objet. Python appelle ensuite `__init__` avec cet objet comme `self` et 3 comme `depart`. L'affectation `self.valeur = depart` lui donne un attribut `valeur` égal à 3.

On parle souvent de « constructeur ». Plus précisément, `__init__` initialise un objet déjà créé. Elle ne doit pas renvoyer de résultat autre que `None` : on n'y écrit pas `return self`.

Source : Python : classes et initialisation

Une méthode agit sur son instance

Une méthode se définit avec `def` dans la classe. Lors d'un appel sur une instance, Python fournit automatiquement cette instance au premier paramètre `self`.

```
class Compteur:
    def __init__(self, depart):
        self.valeur = depart

    def ajouter(self, nombre):
        self.valeur += nombre

c = Compteur(3)
c.ajouter(2)
print(c.valeur)
```

Résultat : 5. L'appel `c.ajouter(2)` revient ici à `Compteur.ajouter(c, 2)`. `self.valeur` appartient à l'objet. `nombre` est un paramètre local de l'appel. Une méthode peut modifier l'état, renvoyer un résultat, ou faire les deux.

À programmer : un produit

Écrivez une classe `Produit`.

- `Produit(nom, prix)` conserve les deux valeurs dans des attributs.
- `total(quantite)` renvoie le prix multiplié par la quantité, sans modifier l'objet.

Créez un cahier à 20 et un stylo à 5. Affichez le nom du cahier, le montant pour trois cahiers et le montant pour deux stylos. Expliquez quel objet `self` désigne dans chaque appel.

Pour l'instant, on suppose les prix valides. Dans les exercices qui suivent, les prix et les taux sont des nombres finis, et les quantités des entiers positifs ou nuls. Nous ajouterons les vérifications au fur et à mesure.

Corrigé : la classe Produit

```
class Produit:
    def __init__(self, nom, prix):
        self.nom = nom
        self.prix = prix

    def total(self, quantite):
        return self.prix * quantite

cahier = Produit("Cahier", 20)
stylo = Produit("Stylo", 5)
print(cahier.nom, cahier.total(3), stylo.total(2))
```

Affichage : Cahier 60 10. Les deux objets ont chacun leurs attributs. Dans `cahier.total(3)`, `self` désigne le cahier. Dans `stylo.total(2)`, il désigne le stylo. La méthode renvoie le calcul avec `return`. Le programme décide ensuite de l'afficher.

Attribut d'instance, attribut de classe et alias

Un attribut créé sur `self` appartient à l'instance. Un attribut écrit directement dans le corps de la classe appartient à la classe et peut servir de valeur commune.

```
class Panier:
    devise = "EUR"

    def __init__(self):
        self.articles = []
```

Chaque appel à `Panier()` crée une nouvelle liste `articles`. `devise` se trouve sur la classe. On peut la lire avec `Panier.devise` ou depuis une instance.

Un alias est un autre nom du même objet : `b = a` ne crée pas de copie. L'expression `a is b` teste si les deux noms désignent exactement le même objet.

À tracer : combien de paniers ?

À partir de la classe précédente, prévoyez les affichages. Dessinez les objets, puis reliez chaque variable à son objet.

```
a = Panier()
b = a
c = Panier()
a.articles.append("Stylo")
print(b.articles)
print(c.articles)
print(a is b, a is c)
```

Déplacez ensuite `articles = []` dans le corps de la classe, à côté de `devise`, et remplacez le contenu de `__init__` par `pass` (ne rien faire). Recréez les paniers. Pourquoi le contenu de `c.articles` change-t-il ?

Corrigé : les objets et la liste partagée

Version	Références	Affichages
Liste créée dans <code>__init__</code>	a et b partagent un panier. c en désigne un autre.	<code>['Stylo']</code> pour b, <code>[]</code> pour c
Liste placée sur la classe	Deux paniers distincts utilisent la même liste de classe.	Les deux affichages donnent la liste contenant Stylo

Dans la première version, `a is b` vaut `True` et `a is c` vaut `False`. Déplacer la liste ne change pas l'identité des paniers, mais change l'endroit où ils trouvent `articles`.

La bonne place pour cette liste : `self.articles = []` dans `__init__`. Une devise commune convient comme attribut de classe. Une liste d'achats doit appartenir à chaque panier.

Encapsulation, getter et setter

L'**encapsulation** consiste à proposer des opérations qui maintiennent l'objet dans un état cohérent. Par exemple, un produit ne doit pas accepter de prix négatif. Un **getter** lit une valeur. Un **setter** la modifie en appliquant les règles prévues.

```
# Methodes a placer dans une classe :
def get_prix(self):
    return self._prix

def set_prix(self, valeur):
    if valeur < 0:
        raise ValueError("prix negatif")
    self._prix = valeur
```

Le préfixe `_` signale un attribut interne par convention. Il n'empêche pas techniquement son accès. `raise ValueError(...)` interrompt l'appel en signalant une valeur invalide. La vérification précède l'affectation pour conserver l'ancien prix en cas de refus.

À programmer : un prix toujours valide

Écrivez `ProduitAcces(nom, prix)` avec les méthodes `get_prix` et `set_prix` définies précédemment. Le constructeur doit passer par `set_prix` afin de vérifier aussi le prix initial.

1. Créez un produit à 20, puis fixez son prix à 25.
2. Essayez un prix de -5 , en traitant l'erreur avec `try/except ValueError`.
3. Lisez le prix après le refus, puis essayez un prix nul.
4. Vérifiez qu'une création avec un prix négatif échoue également.

Avant les essais, notez le prix attendu après chaque opération.

Corrigé : une seule vérification du prix

```
class ProduitAcces:
    def __init__(self, nom, prix):
        self.nom = nom
        self.set_prix(prix)

    def get_prix(self):
        return self._prix

    def set_prix(self, valeur):
        if valeur < 0:
            raise ValueError("prix negatif")
        self._prix = valeur
```

Le prix passe de 20 à 25. Le refus de -5 le laisse à 25. Le prix nul est accepté.

`ProduitAcces("Cahier", -1)` lève `ValueError` dès l'initialisation. Appeler le setter depuis le constructeur évite de recopier la même vérification.

Le symbole @ et les décorateurs

Une fonction est aussi une valeur que l'on peut transmettre à une autre fonction. Un **décorateur** reçoit une fonction et renvoie l'objet qui sera associé à son nom. La ligne `@signaler`, placée juste avant `def`, applique ce décorateur à la définition.

```
def signaler(fonction):  
    print("decoration")  
    return fonction  
  
@signaler  
def bonjour():  
    return "Bonjour"
```

Cela équivaut à définir `bonjour`, puis à écrire `bonjour = signaler(bonjour)`. L'exécution affiche `decoration` au moment de la définition. Un appel ultérieur à `bonjour()` renvoie `"Bonjour"` sans réappliquer le décorateur. Ici, il rend la fonction inchangée. D'autres décorateurs la transforment.

Ce que fait @property

Une **propriété** permet de gérer l'accès à un attribut par des méthodes. `property` est fourni par Python : utilisé comme décorateur, il transforme une méthode de lecture en propriété.

```
class ProduitLecture:
    def __init__(self, prix):
        self._prix = prix

    @property
    def prix(self):
        return self._prix

p = ProduitLecture(20)
print(p.prix)
```

Résultat : 20. Dans la classe, `@property` revient à écrire `prix = property(prix)` après la méthode. La lecture `p.prix` déclenche alors son getter. On ne met pas de parenthèses. La valeur reste stockée dans `_prix`.

Associer un setter à une propriété

Le décorateur `@prix.setter` ajoute une méthode d'écriture à la propriété `prix`. La méthode conserve le même nom pour remplacer la propriété par sa version complète.

```
# Dans la meme classe, apres le getter prix :
@prix.setter
def prix(self, valeur):
    if valeur < 0:
        raise ValueError("prix negatif")
    self._prix = valeur
```

L'affectation `p.prix = 25` appelle ce setter avec `valeur = 25`. Sans setter, une affectation à cette propriété lève `AttributeError` : elle est en lecture seule.

`prix` est le nom public. `_prix` contient la valeur. Pour vérifier aussi le prix initial, le constructeur doit écrire `self.prix = prix` et passer ainsi par le setter.

À programmer : la propriété prix

Réécrivez `Produit` avec un attribut `nom` et une propriété `prix`. Conservez la méthode `total(quantite)`.

- Le prix se lit avec `p.prix` et se modifie avec `p.prix = valeur`.
- La création et les modifications refusent un prix négatif.
- Une erreur laisse le prix précédent inchangé.

Essayez un produit à 20, une modification à 25 et un refus de -5 . Calculez ensuite le montant pour deux produits.

Quel attribut le getter doit-il lire pour éviter de s'appeler lui-même ? Quel nom faut-il utiliser dans le constructeur pour déclencher sa validation ?

Corrigé : la classe Produit avec validation

```
class Produit:
    def __init__(self, nom, prix):
        self.nom = nom
        self.prix = prix
    @property
    def prix(self):
        return self._prix
    @prix.setter
    def prix(self, valeur):
        if valeur < 0:
            raise ValueError("prix negatif")
        self._prix = valeur
    def total(self, quantite):
        return self.prix * quantite
```

Après les essais, le prix vaut 25 et `total(2)` renvoie 50. Cette version complète de `Produit` servira pour la suite. Recréez les instances après l'avoir définie.

À corriger : un setter qui s'appelle lui-même

Dans la classe précédente, remplacez temporairement le setter par cette version. Tracez les trois premiers appels provoqués par `Produit("Cahier", 20)`.

```
@prix.setter
def prix(self, valeur):
    if valeur < 0:
        raise ValueError("prix negatif")
    self.prix = valeur
```

Expliquez ce qui empêche l'initialisation de se terminer, puis corrigez une ligne. Pourquoi déplacer la validation après l'affectation ne réglerait-il pas ce problème ?

Corrigé : le nom public et le stockage

Instruction	Ce qui se passe	Conséquence
<code>self.prix = 20</code>	Appelle le setter	valeur vaut 20
<code>self.prix = valeur</code>	Appelle de nouveau le setter	Le même appel recommence
<code>self._prix = valeur</code>	Stocke directement la valeur	Le setter peut se terminer

Le bug finit par lever `RecursionError`, car les appels s'accumulent. La correction est `self._prix = valeur`, après la vérification.

Une propriété permet de contrôler l'accès sans changer l'écriture `p.prix`. Elle est utile si une règle ou un calcul le justifie. Un attribut ordinaire reste suffisant quand il n'y a rien à contrôler.

L'héritage et la redéfinition d'une méthode

L'**héritage** permet de définir une classe à partir d'une autre. La classe fille reprend les méthodes de la classe mère et peut en **redéfinir** certaines : elle écrit alors une nouvelle méthode portant le même nom.

```
class ProduitSpecial(Produit):  
    pass  
  
p = ProduitSpecial("Cahier", 20)  
print(p.prix, p.total(2))
```

Résultat : 20 40. Les parenthèses de `class ProduitSpecial(Produit)` indiquent la classe mère. `pass` signifie qu'on n'ajoute rien pour le moment. Sans `__init__` dans la classe fille, Python utilise celui du parent, ainsi que sa propriété `prix` et sa méthode `total`.

Le rôle de super()

Dans notre héritage simple, `super()` permet d'appeler une méthode de la classe mère sur l'instance courante. C'est utile pour compléter un comportement existant.

```
class ProduitSolde(Produit):  
    def __init__(self, nom, prix, taux):  
        super().__init__(nom, prix)  
        self._taux = taux
```

Le parent initialise le nom et valide le prix. La classe fille conserve en plus le taux de remise. Un `__init__` redéfini ne déclenche pas automatiquement celui du parent.

Dans une méthode `total` redéfinie, `super().total(quantite)` utilisera le calcul du parent. `self` continuera à désigner le même produit soldé. Aucun nouvel objet n'est créé par cet appel.

À programmer : un produit soldé

Complétez `ProduitSolde(nom, prix, taux)`.

- Le constructeur refuse un taux hors de $[0, 1]$ avec `ValueError`.
- La méthode `total(quantite)` applique la remise au montant calculé par le parent.
- Le calcul ne modifie ni le prix de départ ni le taux.

Un cahier à 20 avec une remise de 25 % doit coûter 30 pour deux unités. Testez également les taux 0 et 1. Répétez le calcul sur le même objet pour vérifier que la remise ne s'accumule pas.

Corrigé : compléter le calcul du parent

```
class ProduitSolde(Produit):
    def __init__(self, nom, prix, taux):
        if not 0 <= taux <= 1:
            raise ValueError("taux invalide")
        super().__init__(nom, prix)
        self._taux = taux

    def total(self, quantite):
        return super().total(quantite) * (1 - self._taux)

p = ProduitSolde("Cahier", 20, 0.25)
print(p.total(2), p.total(2), p.prix)
```

Affichage : 30.0 30.0 20. Le parent calcule 40, puis la classe fille applique le coefficient 0.75. Avec un taux de 0, le montant reste 40. Avec un taux de 1, il devient nul.

Le polymorphisme et le contrat d'une méthode

Le **polymorphisme** permet d'appeler la même opération sur des objets différents, chacun fournissant son propre comportement. `p.total(2)` utilise la méthode correspondant à l'objet que désigne `p`.

Objet	Méthode utilisée	Montant pour deux unités
<code>Produit("Stylo", 5)</code>	<code>Produit.total</code>	10
<code>ProduitSolde("Cahier", 20, 0.25)</code>	<code>ProduitSolde.total</code>	30.0

Un **contrat** précise les entrées acceptées, le résultat et les effets d'une opération. Ici, `total(q)` reçoit une quantité entière positive ou nulle, renvoie un montant numérique positif ou nul, et ne modifie pas l'objet. Un programme peut ainsi utiliser ces objets sans connaître le détail de chaque calcul.

À programmer et tracer : le montant total

Écrivez `montant_total(elements, quantite)` : pour chaque élément, appelez `total(quantite)` et additionnez le résultat. Aucun test sur la classe de l'élément n'est nécessaire.

```
elements = [Produit("Stylo", 5),  
            ProduitSolde("Cahier", 20, 0.25)]  
# montant_total(elements, 2)
```

Donnez le pseudocode, puis tracez l'accumulateur après chaque tour. À quel moment Python choisit-il entre la méthode du produit ordinaire et celle du produit soldé ?

Corrigé : une seule boucle

```
def montant_total(elements, quantite):  
    montant = 0  
    for element in elements:  
        montant += element.total(quantite)  
    return montant
```

Tour	Appel	Accumulateur
Avant la boucle	Aucun	0
1	Stylo : 5 fois 2	10
2	Cahier soldé : (20 fois 2) fois 0.75	40.0

À chaque appel de `element.total(2)`, Python recherche la méthode sur l'objet concerné et sa classe. Pour le produit soldé, cette méthode appelle ensuite celle du parent avec `super()`. Le résultat final est 40.0. Avec une liste vide, il est 0.

Le polymorphisme sans classe mère commune

Python peut utiliser un objet dès qu'il fournit les opérations attendues, même sans héritage commun. Cette manière de programmer s'appelle le *duck typing*.

```
class Service:
    def __init__(self, tarif):
        if tarif < 0:
            raise ValueError("tarif negatif")
        self.tarif = tarif
    def total(self, quantite):
        return self.tarif * quantite

print(montant_total([Produit("Stylo", 5), Service(12)], 2))
```

Résultat : 34. `montant_total` demande seulement une méthode `total(q)` conforme au contrat. Elle n'a besoin ni d'un attribut `prix`, ni d'un nom, ni d'une classe mère particulière.

À corriger : un contrat rompu

Une classe fille doit rester utilisable par les programmes qui attendent un objet de la classe mère. La présence du bon nom de méthode ne suffit pas : le résultat doit aussi respecter le contrat.

```
class ProduitTexte(Produit):
    def total(self, quantite):
        return f"{super().total(quantite)} EUR"

print(montant_total([ProduitTexte("Stylo", 5)], 2))
```

Expliquez pourquoi l'addition échoue. Où placer l'affichage de la devise pour conserver un calcul utilisable ?

Corrigé : calculer, puis mettre en forme

```
p = Produit("Stylo", 5)
montant = montant_total([p], 2)
print(f"{{montant}} EUR")
```

Affichage : 10 EUR. Le calcul renvoie un nombre. Le programme transforme ensuite ce résultat en texte pour l'afficher. La version précédente levait `TypeError`, car elle tentait d'ajouter une chaîne à un nombre.

Une spécialisation doit respecter les entrées, les résultats et les effets annoncés par la classe mère. Cette règle est connue sous le nom de **principe de substitution de Liskov**.

Une classe abstraite pour imposer une opération

Une **classe abstraite** sert de base. Avec le module standard `abc`, on hérite de `ABC` et on marque avec `@abstractmethod` les méthodes que les classes filles devront fournir.

```
from abc import ABC, abstractmethod

class Tarif(ABC):
    @abstractmethod
    def total(self, quantite):
        pass

class TarifUnitaire(Tarif):
    def total(self, quantite):
        return 10 * quantite
```

On ne peut pas créer d'instance tant qu'une méthode abstraite reste à définir : `Tarif()` lève `TypeError`. `TarifUnitaire().total(3)` renvoie 30. L'`ABC` impose la présence de la méthode, mais ne vérifie ni ses paramètres ni son résultat.

Source : Python : classes abstraites

La composition : un objet en utilise d'autres

La **composition** consiste à conserver d'autres objets dans des attributs pour leur confier une partie du travail. Une commande contient des produits. Elle n'est pas un produit et n'a donc pas à en hériter.

```
class Commande:
    def __init__(self, produits, remise):
        self._produits = list(produits)
        self.remise = remise
```

Le constructeur reçoit les objets à utiliser. `list(produits)` copie la liste, mais pas les produits qu'elle contient : les instances restent partagées.

L'héritage sert à spécialiser un type existant en respectant son contrat. La composition sert à répartir le travail entre plusieurs objets que l'on peut éventuellement remplacer.

Le patron de conception Stratégie

Un **patron de conception** est une solution générale à un problème d'organisation du programme. Le patron **Stratégie** consiste à confier un calcul à un objet interchangeable. Ici, la commande confie le calcul de la remise à un objet.

```
class SansRemise:
    def appliquer(self, montant):
        return montant
```

Contrat : `appliquer(montant)` reçoit un montant positif ou nul et renvoie le montant à payer, lui aussi positif ou nul. La commande appellera toujours cette méthode, quelle que soit la remise choisie.

`SansRemise()` n'a aucune donnée à initialiser, donc nous n'écrivons pas de `__init__`. Une fonction peut aussi représenter une stratégie simple. Nous utiliserons des objets pour conserver le taux ou le montant d'une remise.

À programmer : une remise et une commande

1. La remise. Écrivez `Pourcentage(taux)`. Le constructeur vérifie que le taux appartient à $[0, 1]$. La méthode `appliquer(montant)` renvoie le montant après remise.

2. La commande. Complétez `Commande` avec `total()`. Chaque produit de la liste représente une unité. Additionnez les `produit.total(1)`, puis transmettez la somme à `self.remise.appliquer`.

Utilisez deux produits à 20 et 30. Prévoyez le résultat sans remise, puis avec une remise de 10 %. Tracez la somme après chaque tour de boucle. Essayez aussi une commande vide.

Corrigé : le calcul du pourcentage

```
class Pourcentage:
    def __init__(self, taux):
        if not 0 <= taux <= 1:
            raise ValueError("taux invalide")
        self._taux = taux

    def appliquer(self, montant):
        return montant * (1 - self._taux)

print(SansRemise().appliquer(50))
print(Pourcentage(0.10).appliquer(50))
```

Résultats : 50 et 45.0. Un taux nul conserve le montant. Un taux de 1 ramène le montant à zéro. La classe conserve le taux dans son état et l'utilise à chaque appel, sans modifier le montant reçu.

Corrigé : la commande utilise la remise

```
class Commande:
    def __init__(self, produits, remise):
        self._produits = list(produits)
        self.remise = remise

    def total(self):
        somme = 0
        for produit in self._produits:
            somme += produit.total(1)
        return self.remise.appliquer(somme)

produits = [Produit("Cahier", 20), Produit("Livre", 30)]
c = Commande(produits, Pourcentage(0.10))
print(c.total())
```

Trace : la somme vaut 0, puis 20, puis 50. La remise donne 45.0. Sans remise, le résultat est 50. Une commande vide donne 0 avec ces deux stratégies.

À essayer : changer de remise

Ajoutez `RemiseFixe(reduction)` avec la méthode `appliquer(montant)`. Refusez une réduction négative. Le montant à payer doit rester positif ou nul, même si la réduction le dépasse.

Essayez une réduction de 7 sur 50, puis de 80 sur 50. Remplacez ensuite la remise de la commande existante par `SansRemise()`.

Avez-vous dû modifier la classe `Commande` ? Quelles conditions les objets de remise doivent-ils respecter pour permettre ce remplacement ?

Corrigé : une nouvelle remise

```
class RemiseFixe:
    def __init__(self, reduction):
        if reduction < 0:
            raise ValueError("reduction negative")
        self._reduction = reduction
    def appliquer(self, montant):
        return max(0, montant - self._reduction)

c = Commande(produits, RemiseFixe(7))
print(c.total())
c.remise = SansRemise()
print(c.total())
print(RemiseFixe(80).appliquer(50))
```

Résultats : 43, 50, puis 0. Commande reste inchangée. Chaque objet de remise propose la même méthode appliquer et respecte son contrat.

Le patron Adaptateur : réutiliser une autre interface

L'**interface** d'un objet désigne les opérations par lesquelles on l'utilise. Un **adaptateur** présente l'interface attendue et traduit les appels vers un objet existant.

```
class AncienCoupon:
    def calculer_net(self, montant):
        return max(0, montant - 7)
```

La commande attend appliquer. L'ancien outil propose calculer_net. Avec un AncienCoupon() fourni directement, le calcul du total lèverait AttributeError.

À écrire : AdaptateurCoupon(ancien) conserve l'objet reçu. Sa méthode appliquer(montant) appelle ancien.calculer_net(montant). Réutilisez le calcul existant et essayez l'adaptateur sur la commande de 50.

Corrigé : l'adaptateur traduit l'appel

```
class AdaptateurCoupon:
    def __init__(self, ancien):
        self._ancien = ancien

    def appliquer(self, montant):
        return self._ancien.calculer_net(montant)

remise = AdaptateurCoupon(AncienCoupon())
print(Commande(produits, remise).total())
```

Résultat : 43. La commande appelle l'adaptateur, qui appelle l'ancien coupon et lui transmet 50. Ni la commande ni le calcul du coupon n'ont besoin d'être modifiés.

La stratégie permet de choisir un calcul. L'adaptateur permet de réutiliser un objet dont l'interface est différente. Les deux reposent ici sur la composition.

Choisir une organisation adaptée

Besoin	Choix possible	Justification
Un calcul sans état	Fonction	Les arguments suffisent au calcul.
Des données avec des règles	Classe et propriétés	Les opérations préservent la cohérence de l'objet.
Une spécialisation	Héritage	La classe fille respecte le contrat du parent.
Un calcul interchangeable	Stratégie	L'appelant utilise toujours la même opération.
Une interface différente	Adaptateur	Il traduit les appels vers l'objet existant.

Créer davantage de classes ne rend pas automatiquement le programme plus clair. Choisissez une classe quand elle rassemble un état et des opérations utiles, ou quand elle représente un rôle précis dans le programme.

En data science : apprendre puis réutiliser

Un prétraitement prépare les données avant leur utilisation par un modèle. Les données d'apprentissage servent à calculer ses paramètres. Pour **centrer** des valeurs, on calcule une moyenne et on la soustrait à chaque valeur.

Les bibliothèques utilisent souvent deux noms de méthodes, qui sont des **conventions** et non des mots-clés de Python.

Méthode	Rôle	Pour un centreur
<code>fit(donnees)</code>	Calcule et mémorise les paramètres à partir des données d'apprentissage.	Mémorise la moyenne de [2, 4, 6], soit 4.
<code>transform(donnees)</code>	Applique le traitement avec les paramètres déjà calculés.	Soustrait cette même moyenne à de nouvelles valeurs.

L'objet permet de conserver cette moyenne entre les appels. `fit` renverra ici `self`, par convention, pour autoriser l'enchaînement des appels.

À programmer : un centreur de données

Écrivez `Centreur` avec un attribut `_moyenne` initialement égal à `None`, pour indiquer qu'aucune moyenne n'a encore été calculée.

- `fit(valeurs)` refuse une liste vide, mémorise la moyenne et renvoie `self`.
- `transform(valeurs)` renvoie une nouvelle liste contenant chaque valeur moins la moyenne.
- Un appel à `transform` avant `fit` lève `RuntimeError("fit requis")`.

Les valeurs sont des nombres finis. Les méthodes ne modifient pas la liste reçue. Après apprentissage, transformer une liste vide donne une liste vide.

Écrivez les pseudocodes. Avec `[2, 4, 6]` pour l'apprentissage, tracez la transformation de `[2, 4, 6]`, puis de `[10, 12]`.

Corrigé : calculer et conserver la moyenne

```
class Centreur:
    def __init__(self):
        self._moyenne = None
    def fit(self, valeurs):
        if not valeurs:
            raise ValueError("liste vide")
        self._moyenne = sum(valeurs) / len(valeurs)
        return self

c = Centreur()
retour = c.fit([2, 4, 6])
print(retour is c)
```

La moyenne vaut 4.0 et l'affichage est True. `fit` renvoie l'objet lui-même : `Centreur().fit(...)` reste donc un centreur. Contrairement à `__init__`, cette méthode peut renvoyer `self`.

Corrigé : transformer avec la moyenne conservée

Ajoutez cette méthode dans Centreur, au même niveau que fit.

```
def transform(self, valeurs):  
    if self._moyenne is None:  
        raise RuntimeError("fit requis")  
    resultat = []  
    for valeur in valeurs:  
        resultat.append(valeur - self._moyenne)  
    return resultat
```

Entrée	Liste après chaque tour	Moyenne utilisée
[2, 4, 6]	[-2.0], puis [-2.0, 0.0], puis [-2.0, 0.0, 2.0]	4.0
[10, 12]	[6.0], puis [6.0, 8.0]	4.0

transform utilise la moyenne sans la recalculer. Un nouvel appel à fit la remplacerait.

À comparer : quelles données servent à apprendre ?

Le jeu d'apprentissage sert à ajuster les traitements et le modèle. Le jeu de test sert ensuite à évaluer le résultat sur d'autres données. Prévoyez les deux sorties suivantes.

```
train = [2, 4, 6]
test = [10, 12]

c1 = Centreur().fit(train)
print(c1.transform(test))

c2 = Centreur().fit(train)
c2.fit(test)
print(c2.transform(test))
```

Quel appel change la moyenne du second centreur ? Le traitement utilise-t-il encore uniquement les informations du jeu d'apprentissage ?

Corrigé : conserver les paramètres d'apprentissage

Objet	Moyenne utilisée sur test	Sortie
c1	4, calculée sur train	[6.0, 8.0]
c2	11, recalculée sur test	[-1.0, 1.0]

`c2.fit(test)` remplace la moyenne précédente. Le traitement dépend maintenant des données qui devaient servir à l'évaluation. C'est une **fuite d'information** : des informations du jeu de test interviennent dans l'apprentissage.

Pour évaluer un modèle, on ajuste le prétraitement sur le jeu d'apprentissage, puis on applique les mêmes paramètres au jeu de test. La séparation entre `fit` et `transform` aide à respecter cette règle.

Source : scikit-learn : data leakage

Un pipeline compose plusieurs traitements

Un **pipeline** applique plusieurs étapes dans un ordre donné. La sortie d'une étape devient l'entrée de la suivante. Nous allons combiner le centreur avec une multiplication par un facteur.

```
class Multiplier:
    def __init__(self, facteur):
        self.facteur = facteur
    def fit(self, valeurs):
        return self
    def transform(self, valeurs):
        return [v * self.facteur for v in valeurs]
```

Le facteur est choisi à la création. Cette étape n'a rien à apprendre sur les données, donc `fit` renvoie simplement l'objet. Elle offre néanmoins les mêmes méthodes que `Centreur`. Le pipeline pourra appeler `fit` et `transform` sur chacune de ses étapes.

À programmer : un pipeline simple

Écrivez `PipelineSimple(etales)`. Le constructeur conserve une copie de la liste d'étapes.

- `fit(valeurs)` parcourt les étapes. Chaque étape apprend sur les valeurs courantes, puis les transforme pour la suivante. La méthode renvoie le pipeline.
- `transform(valeurs)` parcourt les étapes en utilisant seulement leur méthode `transform`.
- Chaque parcours part d'une copie de la liste de valeurs.

Écrivez le pseudocode des deux boucles. Utilisez [`Centreur()`, `Multiplier(10)`] avec [2, 4, 6] pour l'apprentissage, puis [10, 12] pour la transformation. Notez les valeurs après chaque étape. Créez de nouvelles instances d'étapes pour chaque pipeline.

Corrigé : les deux parcours du pipeline

```
class PipelineSimple:
    def __init__(self, etapes):
        self.etapes = list(etapes)
    def fit(self, valeurs):
        courant = list(valeurs)
        for etape in self.etapes:
            etape.fit(courant)
            courant = etape.transform(courant)
        return self
    def transform(self, valeurs):
        courant = list(valeurs)
        for etape in self.etapes:
            courant = etape.transform(courant)
        return courant
```

Chaque étape apprend sur la sortie de la précédente. La boucle de transform n'appelle jamais fit, donc elle ne recalcule aucun paramètre.

Corrigé : le suivi des valeurs

Parcours	Étape	Valeurs produites
<code>fit([2, 4, 6])</code>	Centreur, moyenne 4	<code>[-2.0, 0.0, 2.0]</code>
Suite de <code>fit</code>	Multiplier, facteur 10	<code>[-20.0, 0.0, 20.0]</code>
<code>transform([10, 12])</code>	Centreur, moyenne 4	<code>[6.0, 8.0]</code>
Suite de <code>transform</code>	Multiplier, facteur 10	<code>[60.0, 80.0]</code>

La transformation renvoie `[60.0, 80.0]`. Les listes d'entrée restent inchangées. Un pipeline sans étape renverrait une copie de la liste reçue.

La copie de `etapes` ne copie pas les objets qu'elle contient. Partager un même centreur entre deux pipelines partagerait donc aussi sa moyenne. On retrouve les notions de référence, d'état et de composition étudiées plus haut.

Retrouver ces objets dans scikit-learn

scikit-learn est une bibliothèque d'apprentissage automatique. L'exemple suivant utilise trois classes. Vous pouvez en suivre le fonctionnement sans installer la bibliothèque.

Classe	Rôle	Méthodes utilisées
StandardScaler	Centre les valeurs. Avec <code>with_std=False</code> , il ne divise pas par l'écart-type.	<code>fit</code> , <code>transform</code>
LinearRegression	Apprend une relation affine entre les entrées et une valeur à prédire.	<code>fit</code> , <code>predict</code>
Pipeline	Enchaîne les traitements, puis le modèle final.	<code>fit</code> , <code>predict</code>

`predict` calcule des prédictions avec le modèle déjà ajusté. `X` contient une ligne par observation. `y` contient les valeurs à prédire. Le constructeur de `Pipeline` reçoit une liste de couples (nom, objet).

À lire, puis à essayer : un modèle complet

Repérez la création des objets, leur apprentissage et leur utilisation. Les exemples d'apprentissage suivent la relation $y = 2x + 1$. Prévoyez les deux prédictions.

```
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import Pipeline

modele = Pipeline([
    ("centrage", StandardScaler(with_std=False)),
    ("regression", LinearRegression()),
])
X_train, y_train = [[2], [4], [6]], [5, 9, 13]
X_test = [[10], [12]]
modele.fit(X_train, y_train)
print(modele.predict(X_test).round(6).tolist())
```

`round(6).tolist()` sert uniquement à afficher les prédictions arrondies sous forme de liste.

Corrigé : les paramètres restent dans les objets

Prédictions : [21.0, 25.0]. Pour les entrées 10 et 12, la relation $y = 2x + 1$ donne bien 21 et 25.

Objet	Pendant fit	Pendant predict
StandardScaler	Calcule la moyenne 4	Soustrait la même moyenne
LinearRegression	Apprend sur les entrées centrées	Prédit à partir des entrées centrées
Pipeline	Organise les apprentissages	Applique les traitements, puis le modèle

Les paramètres calculés restent dans les instances. La composition permet d'assembler les étapes, et leurs interfaces permettent de remplacer un traitement ou un modèle. Notre `PipelineSimple` ne contenait que des transformations. Celui-ci se termine par un modèle qui prédit.

Source : scikit-learn : Pipeline

Autoévaluation

Vous devez maintenant pouvoir expliquer un programme objet en suivant ses appels.

- Quand Python appelle-t-il `__init__` ? Que désigne `self` ?
- Que fait un décorateur ? Qu'ajoutent `@property` et `@prix.setter` ?
- Quels attributs appartiennent à l'instance ? Quels objets sont partagés ?
- Quelle méthode est choisie dans une classe fille ? Que fait `super()` ?
- Pourquoi un objet peut-il en remplacer un autre ? Quel contrat doit-il respecter ?
- Quelles données modifient les paramètres d'un centreur ? Que conserve-t-il entre deux appels ?

Pour vérifier une réponse, reprenez l'exemple correspondant, annoncez le résultat attendu, puis exécutez-le. Si vous vous êtes trompé, identifiez l'appel ou l'affectation qui explique la différence.